Music Course Platform
Group 6

Scrum Master: Yicheng CHEN

Lu Liu – Luo Ying – Su Wai Phyoe

Software Engineering Project 1

Metropolia University of Applied Sciences

Information and communication technologies

Software Engineering

Software project

25.1.2026

Music Course Platform

1.Project Overview

1.1 Problem Summary

Finding and booking private music lessons is often difficult and time-consuming for both learners and teachers. Existing methods are fragmented and inefficient, making it hard for talented teachers to reach enough students and for learners to find suitable lessons.

1.2 Intended Audience / Users

Our product is designed for two main user groups:

Music Teachers:

Music academy students and freelance musicians who want to teach instruments in their free time. They need a simple way to present their profile, publish available time slots, set lesson prices, and manage bookings.

Learners (Students / Adults / Parents):

Children, teenagers, and adults who want to learn musical instruments. They need an easy and reliable way to find suitable teachers based on instrument, availability, location, and price, and to book lessons online.

1.3 Main Features / Components

1. Teachers and learners can create accounts and log in securely.

2. Teachers can create and update their profiles, including biography, instruments taught, teaching experience, and pricing.

3. Teachers can publish available time slots for lessons and manage their schedules.

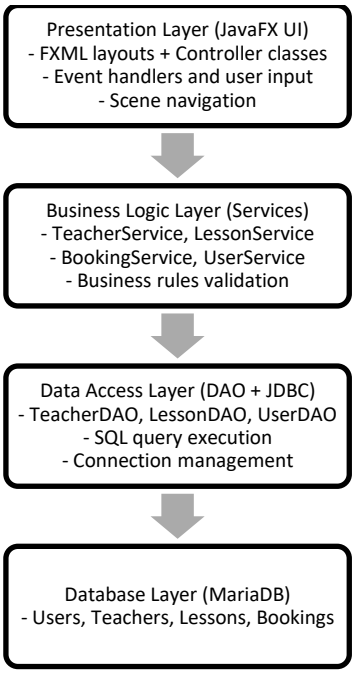
4. Learners can search for teachers by instrument and availability.

5. Learners can book lessons and confirm schedules.

6. Basic notifications for upcoming lessons and booking confirmations.

1.4 System Architecture

This project follows a three-tier architecture pattern:



All layers run within a single Java desktop application.

Communication between layers happens through direct method calls.

2.Project Objectives

2.1 Key Objectives (SMART: Specific, Measurable, Achievable, Relevant)

1. **User Management (Week 1-4)**
    - **Specific**: Implement secure user registration and login for teachers and learners
    - **Measurable**: 100% of registered users can log in successfully with valid credentials

- **Achievable**: Using JavaFX forms and MariaDB user table with password hashing
- **Relevant**: Foundation for all other features
- **Time-bound**: Complete by end of Sprint 2 (Week 4)

2. **Teacher Profile System (Week 3-4)**
   - **Specific**: Enable teachers to create and update complete profiles
   - **Measurable**: Teachers can add/edit at least 5 profile fields (name, bio, instruments, experience, pricing)
   - **Achievable**: Using JavaFX profile forms with JDBC CRUD operations
   - **Relevant**: Allows learners to find suitable teachers
   - **Time-bound**: Complete by end of Sprint 2 (Week 4)
3. **Lesson Booking System (Week 4-6)**
   - **Specific**: Implement functional lesson booking workflow
   - **Measurable**:
     - Teachers can publish at least 10 time slots per week
     - Learners can successfully book 95%+ of available slots
     - Zero double-booking incidents in testing
   - **Achievable**: Using time slot table with booking status management
   - **Relevant**: Core value proposition of the platform
   - **Time-bound**: Basic booking by Week 4, full workflow by Week 6
4. **Search Functionality (Week 5-6)**
   - **Specific**: Multi-criteria teacher search
   - **Measurable**: Users can search by at least 2 criteria (instrument + availability)
   - **Achievable**: Using SQL SELECT with WHERE clauses
   - **Relevant**: Helps learners find suitable teachers quickly

    - **Time-bound**: Complete by end of Sprint 3 (Week 6)

5. **System Quality (Week 7-8)**
    - **Specific**: Achieve stable, bug-free core functionality
    - **Measurable**:
        - Pass 100% of critical integration tests
        - Zero critical bugs (crashes, data loss)
        - Maximum 5 minor UI/UX bugs
        - At least 60% code coverage for business logic
    - **Achievable**: Through systematic JUnit and integration testing
    - **Relevant**: Ensures reliable user experience
    - **Time-bound**: Complete by end of Sprint 4 (Week 8)

3.Scope and Deliverables

3.1 In-Scope Features

- User registration and login for teachers and learners
- Teacher profile management (biography, instruments, pricing)
- Publishing and managing lesson time slots
- Booking lessons and confirming schedules
- Basic notifications and reminders
- JavaFX UI and MariaDB integration (three-tier architecture)
- CRUD functionality for core data: teachers, learners, lessons

3.2 Out-of-Scope Features

- Advanced payment processing – Online payment integration with gateways like Stripe or PayPal, including transaction handling and refund management.

- Comprehensive review and rating system – Full-featured feedback system with ratings, reviews, moderation, and reporting capabilities.
- Advanced notification system – Real-time push notifications, automated email reminders, and SMS alerts require external services and API integrations beyond the course scope.

3.3 Deliverables

- Working Music Course Platform (JavaFX + JDBC + MariaDB)
- Database schema with tables and relationships
- Functional user interface for teachers and learners
- Technical documentation (database, code)
- User manual
- Test reports (unit, integration, system)
- Sprint review reports after each sprint

4.Project Timeline

4.1 Timeline Overview (Gantt / Sprint Table)

Project Management Platform:

We will use Trello for sprint planning, task tracking, and backlog management.

Board Structure:

- Backlog
- To Do (Sprint tasks)
- In Progress
- Review
- Done

Trello Board Link: https://trello.com/b/YnjfjBxd/sep1musiccourseplatform

Update Frequency: Daily standups + Sprint planning meetings Sprint Timeline Table:

Sprint	Weeks	Main Tasks / Deliverables
Sprint1	Week 1-2	Form Project Team Select Project topic Create Project Vision and Project Plan Technology selection Set up Project Management Tools Create initial product backlog Prepare topic presentation Return deliverables and prepare for sprint 1 report
Sprint2	Week 3-4	database schema implemanattion Initial user interface screens Unit tests and test results Maven configuration Publicly accessible JaCoCo code coverage report Updated project repository and sprint board
Sprint3	Week 5-6	Extend core application features based on the product backlog Implement authentication and basic authorization Improve backend logic and data validation Enhance search and filtering functionality Integrate Jenkins for CI/CD (build, test, coverage) Expand unit tests and generate JaCoCo coverage reports Create and test a basic Docker image locally Fix bugs and improve overall system stability

Sprint4	Week 7-8	Complete and stabilize all planned functionalities Containerize the application using Docker Publish the Docker image to Docker Hub Integration testing System testing and bug fixes Code coverage analysis Final UI polish Technical documentation User manual Test reports Final presentation preparation Deliverable: Sprint 4 Review + Final Report

4.2 Phases and Milestones

Requirement Gathering

- Identify user roles (teachers, learners)
- Define core features (registration, profile management, booking)
- Document functional requirements

Design / Database & UI

- Design relational database schema (tables, primary/foreign keys)
- Create basic JavaFX UI forms for login, registration, profile management
- Define navigation flow and screen layouts

Development / Implementation

- Implement CRUD operations for core data
- Connect JavaFX UI with MariaDB via JDBC
- Implement basic notifications and reminders

Testing / QA

- Unit testing for core functionalities
- Integration testing between UI and database
- System testing to ensure stable booking workflow

Deployment / Presentation

- Prepare a working demo of the system
- Write technical documentation and user manual
- Submit Sprint review reports and final deliverables

4.3 Task Dependencies

Database schema design must be completed before CRUD implementation. UI design should be completed before integrating database functions. CRUD functionality must be implemented before adding notifications or reminders. Testing can only begin after basic CRUD and UI integration. Final presentation and documentation depend on completion of all previous phases

5.Resource Allocation

5.1 Team Members and Roles

Team Member	Role / Responsibilities
Luo Ying	Project planning, project plan document, user manual, front-end design and coding
Chen Yicheng	Scrum Master, project schedule management, back-end development, deployment
Lu Liu	Back-end development, technical documentation, database design, presentation slides

Su Wai Phyoe	Testing reports, Project Planning, front-end design and coding, database, presentation slides

5.2 Software, Hardware, and Tools

Software:

- Java Development Kit (JDK 17 or later)
- JavaFX SDK 17+ (for desktop UI)
- JDBC Driver for MariaDB
- MariaDB Server 10.6+ (database)

Build Tools:

- Maven or Gradle (for dependency management)

Version Control:

- Git + GitHub

Testing:

- JUnit 5 (unit testing)
- TestFX (JavaFX UI testing)

Project Management:

- Trello (sprint planning)

5.3 External Resources or Support

Online documentation and tutorials for JavaFX, JDBC, and MariaDB.

Community forums (Stack Overflow, GitHub) for troubleshooting coding or integration issues.

Optional external libraries if needed for UI enhancements or scheduling functions

6.Risk Management

Risk Description	Likelihood	Impact	Mitigation Strategies
Database design or integration issues (MariaDB + JDBC)	Medium	High	Plan database schema early. Perform incremental testing of CRUD operations; consult documentation or instructor when needed
Feature scope creep	Medium	Medium	Define clear In-Scope and Out-of-Scope features; monitor backlog; only implement features planned in sprints
Team member availability or coordination problems	Medium	Medium	Use Scrum meetings, Trello/Jira for task tracking; maintain clear responsibilities and deadlines
System bugs or crashes during testing	Medium	High	Implement unit testing (JUnit) and integration testing; maintain a bug tracking log; fix issues incrementally per sprint
Deployment issues (running JavaFX + MariaDB on team machines)	Low	Medium	Test deployment environment early; provide instructions for running system; standardize development setup
Loss of data or corrupted database	Low	High	Regular backups of MariaDB database; use version control for SQL scripts; test migration procedures

7.Testing and Quality Assurance

7.1 Types of Testing

Unit Testing

Test individual functions and methods in the system (e.g., booking logic, CRUD operations, user authentication)

Integration Testing

Test interactions between the JavaFX UI, JDBC, and MariaDB database

Ensure data flows correctly between front-end and database

User Acceptance Testing (UAT)

Test the system from the perspective of teachers and learners

Verify that registration, booking, notifications, and basic search work as expected.

7.2 Tools and Frameworks for Testing

JUnit – for unit testing of Java classes and methods

TestFX– for automated UI testing of JavaFX interfaces

Manual testing – for user acceptance testing scenarios

Database testing – SQL queries to verify CRUD operations and data integrity

8.Documentation and Reporting

8.1 Project Documentation

- User Manuals

User manuals will guide teachers and learners on using the system for registration, profile management, and lesson booking.

- Technical Documentation

Technical documentation will describe the system architecture, database schema, JavaFX UI structure, and code modules.

- Test documentation

Test documentation will summarize unit, integration, and user acceptance test results.

8.2 Progress Reporting

- **Weekly Updates**

  Weekly updates will track completed tasks and progress.

- **Sprint Review Meetings**

  Discuss completed sprint work, demonstrate features, gather feedback.

- **Reports**

  Sprint review reports submitted in Oma. Final project report and presentation documentation.